

Programmazione 1

Corso c

24/10/2023
> Gli array

Alessandro Mazzei

alessandro.mazzei@unito.it

Dipartimento di Informatica
Università degli Studi di Torino



UNIVERSITÀ
DI TORINO

Outline

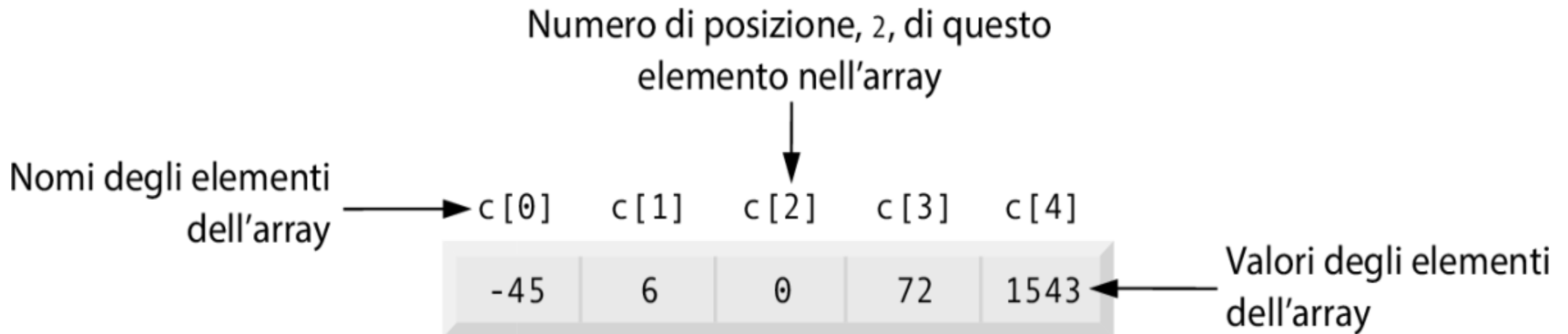
- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Gli Array

- Un array è un gruppo di elementi dello stesso tipo immagazzinati in modo contiguo nella memoria.
- Il diagramma seguente mostra un array di interi chiamato c, contenente cinque elementi:



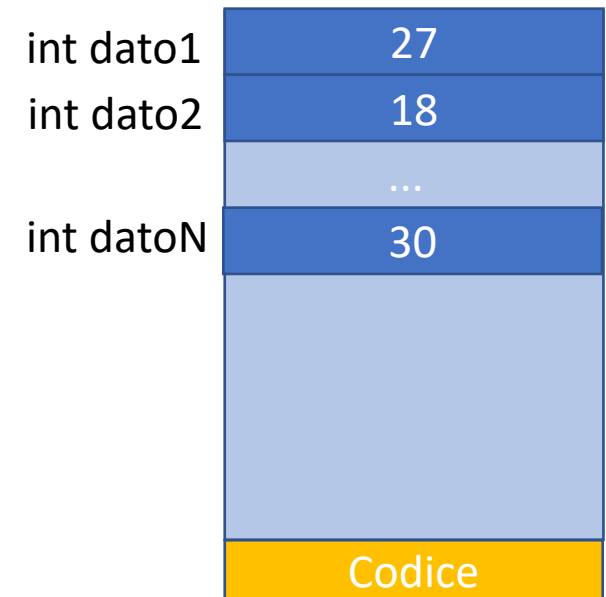
Gli Array

- È spesso necessario memorizzare collezioni di dati omogenei per elaborarle in seguito (statistica, ordinamento, selezione, etc.)
 - I voti di una sessione di esame
 - Le temperature di un periodo
 - L'andamento di un titolo in borsa
- Non ci interessa dare un nome ad ogni elemento
- È sufficiente allocare N elementi ed accedere ad ogni i -esimo elemento della collezione

int dato1	27
int dato2	18
	...
int datoN	30
	Codice

Gli Array

- Gli array sono collezioni di elementi dello stesso tipo allocati in aree contigue di memoria
- Il linguaggio C dispone di sintassi per **allocare** l'array ed **accedere** all'i-esimo elemento
- Per ora ci concentreremo sugli array di dimensione nota al momento della compilazione e allocati nel frame della funzione *main()*

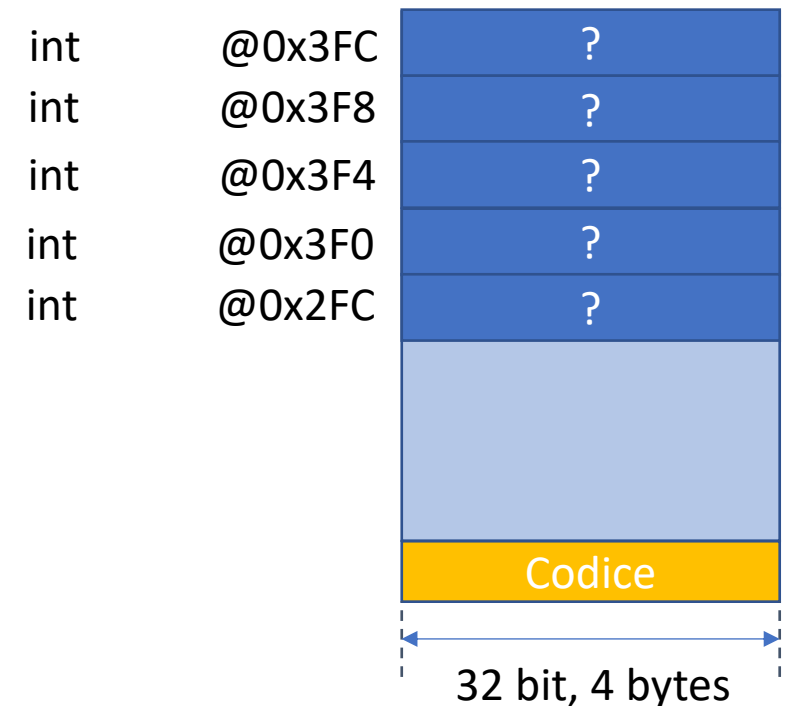


Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Allocazione di un array

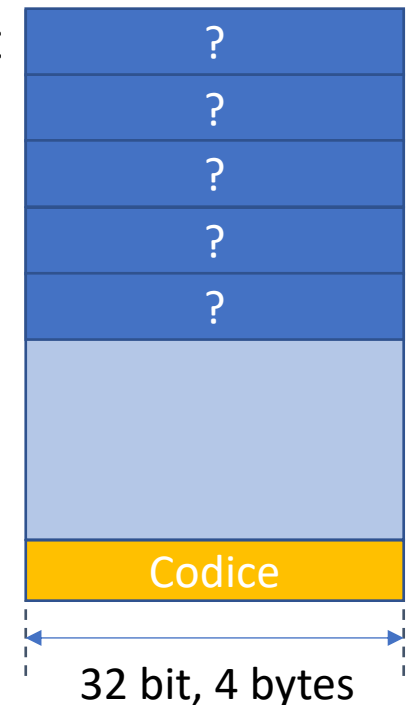
- Un array di N elementi si alloca come
 <tipo> <nome array>[<numero elementi>]
- Per esempio, per allocare un array di nome voti di 5 int nel frame del main(), scriveremo
 int voti[5] ;
- Ciò risulta nell'allocazione di 5 parole da 4 bytes l'una nel frame di main() ad indirizzi consecutivi
- Il contenuto di tali parole è indefinito



Allocazione di un array

- L'accesso al valore dell' i -esimo elemento dell'array avviene in lettura e scrittura come
`<nome array>[<indice>]`
- **L'indice va da 0 a N-1** (nell'esempio, da 0 a 4)
- L'indice 0 corrisponde sempre all'indirizzo più basso (verso la testa dello stack)

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
```



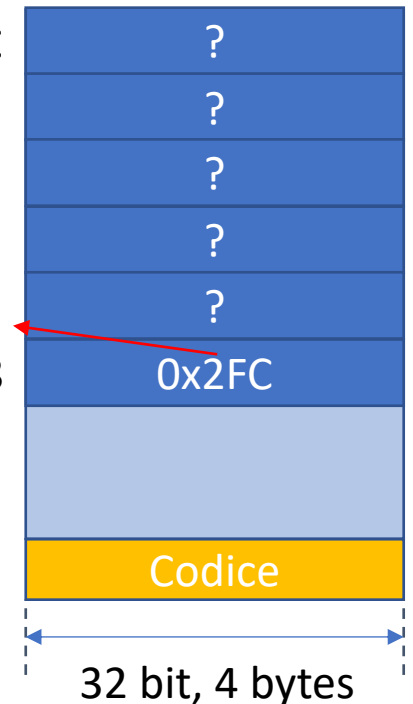
Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Allocazione di un array

- L'accesso al valore dell' i -esimo elemento dell'array avviene in lettura e scrittura come
`<nome array>[<indice>]`
- **L'indice va da 0 a N-1** (nell'esempio, da 0 a 4)
- L'indice 0 corrisponde sempre all'indirizzo più basso (verso la testa dello stack)

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
int* pArr@0x2E8



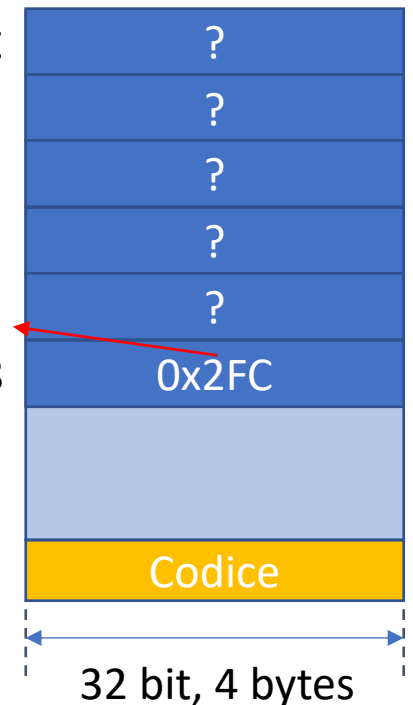
Array e puntatori

- Il nome dell'array è un puntatore che corrisponde all'indirizzo dell'elemento di indice 0

```
int* pArr = voti
```

- L'indirizzo degli elementi successivi è calcolato come l'indirizzo dell'elemento di indice 0 più l'indice moltiplicato per la dimensione del tipo in bytes, perciò gli array sono di tipo omogeneo, contigui in memoria a partire dall'indirizzo più basso e l'indice parte da 0 e non da 1
- Il compilatore C solleva il programmatore dalla gestione della memoria a basso livello con le []

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC  
int* pArr@0x3E8
```



Outline

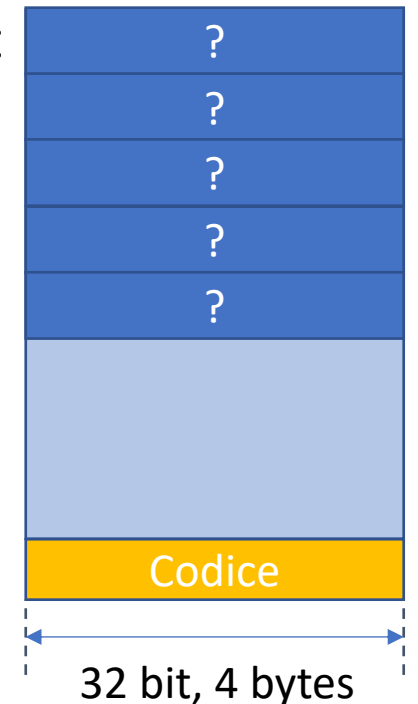
- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Assegnazione di valore

- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
```



Assegnazione di valore

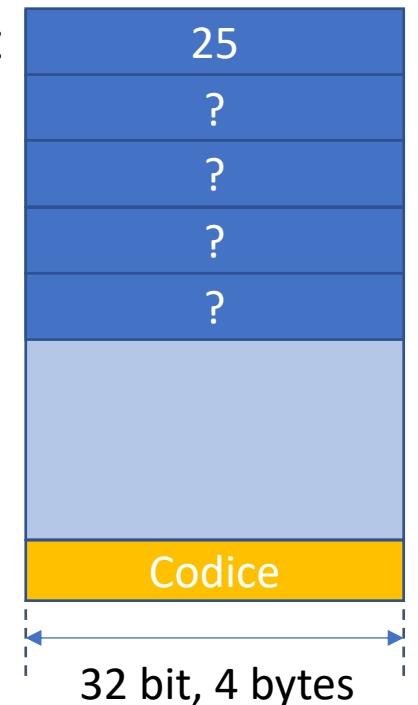
- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

per esempio

```
voti[4]=25;
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```



Assegnazione di valore

- Assegnazione di un valore all'elemento i -esimo

`<nome array>[<indice>] = <valore>`

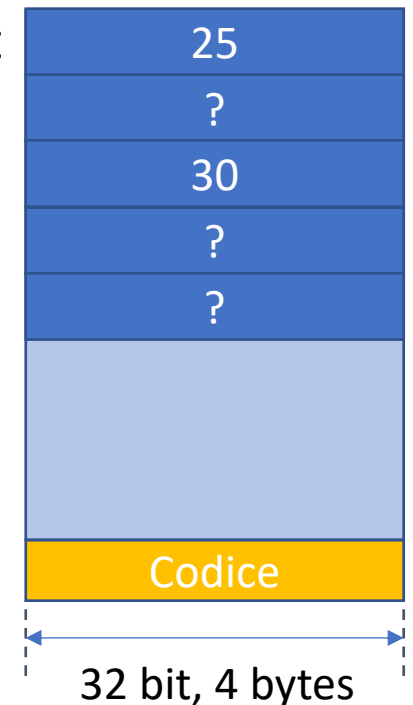
per esempio

```
voti[4]=25;
```

- È possibile accedere all'indirizzo dell'elemento i -esimo tramite referenziazione (altri modi possibili)

```
printf("Inserisci un voto: ");  
scanf("%d ", &voti[2]);
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```



```
$ ./leggi_voto  
Inserisci un voto : 30
```

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Lettura di valore

- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```



Lettura di valore

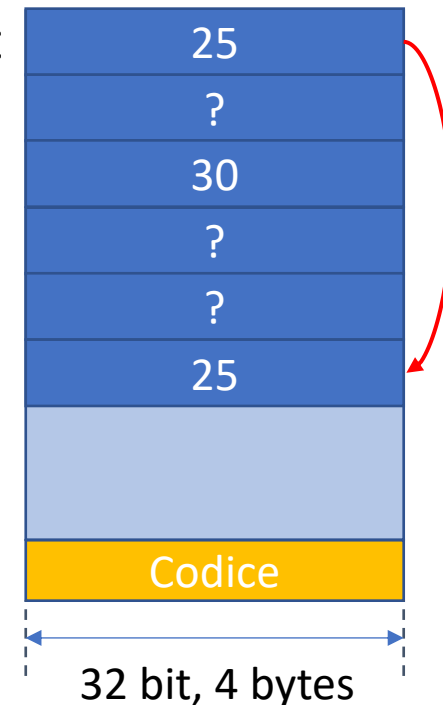
- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

- per esempio

```
int ultVoto = voti[4];
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC  
int ultVoto@0x3E8
```



Lettura di valore

- Lettura valore all'elemento i -esimo e assegnamento

`<variabile> = <nome array>[<indice>]`

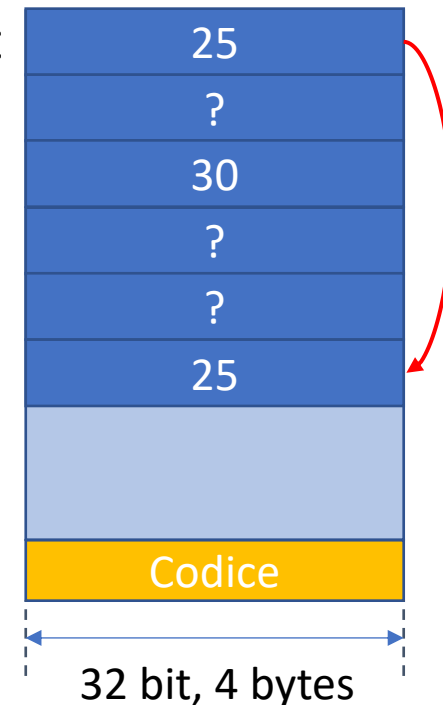
- per esempio

```
int ultVoto = voti[4];
```

- oppure

```
printf("voto: %d", voti[4]);
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
int ultVoto@0x3E8
```



Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Accesso tramite indice

- Spesso si vuole eseguire la stessa operazione (memorizzazione input da tastiera, elaborazione, stampa) su ogni elemento dell'array
- Tipicamente, l'indice dell'elemento non è un valore costante ma una variabile indice che viene fatta variare da 0 a N-1
- Per questa variabile indice utilizzeremo il tipo *int*
 - Il libro utilizza `size_t` , equivalente ad un *unsigned int*
- L'indice viene fatto variare da 0 a N-1
- Definiremo la dimensione dell'array con una `#define` per ora

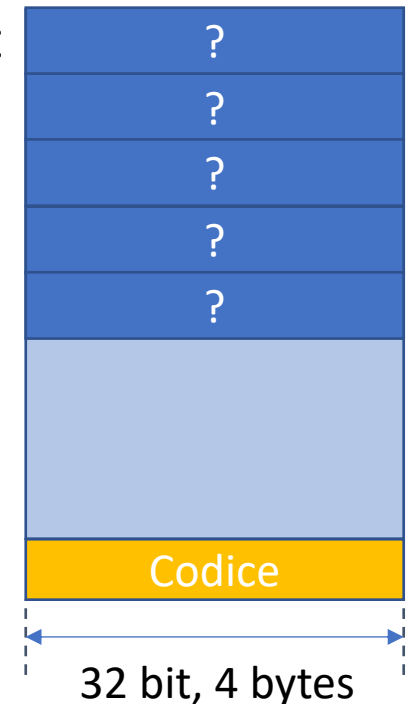
Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    ➔ int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];
```



```
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```

```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC

int i



Accesso tramite indice

```
#define SIZE_A 5  
int main(void) {  
    int voti[SIZE_A];
```

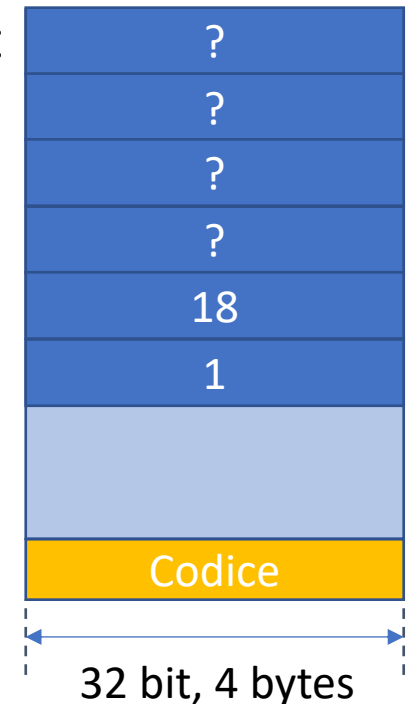


```
    for (int i = 0; i < SIZE_A; i++) {  
        voti[i] = i + 18;  
    }
```

```
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, voto[i]);  
    }  
}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

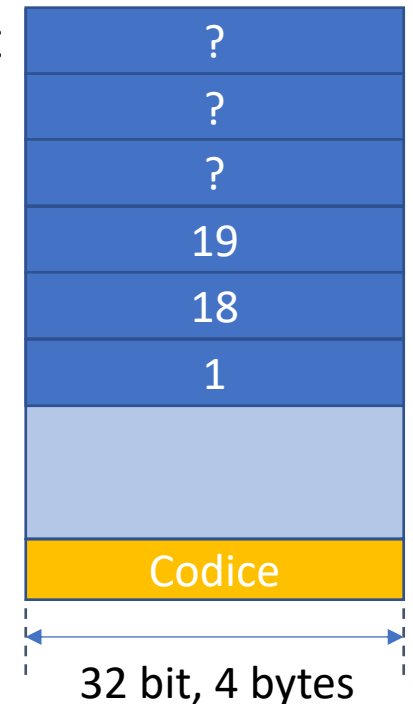
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

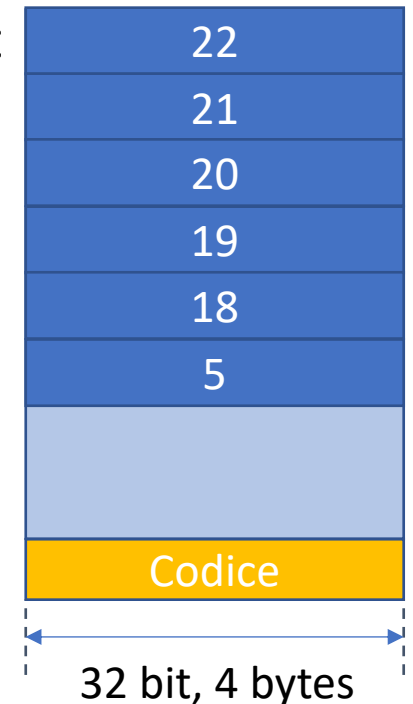
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```



```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

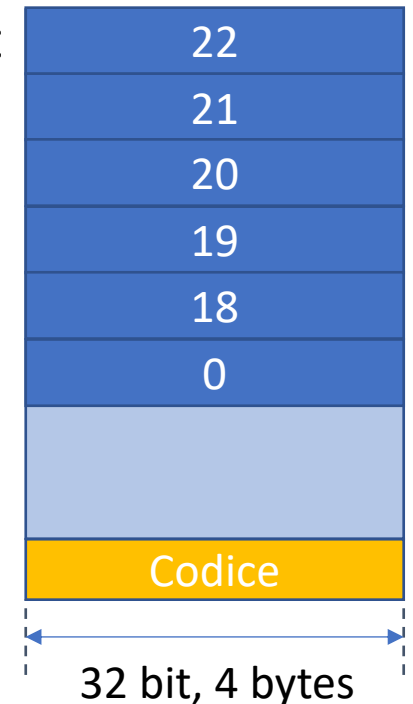
    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }
```

➔

```
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```

```
int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC
```

int i



Accesso tramite indice

```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

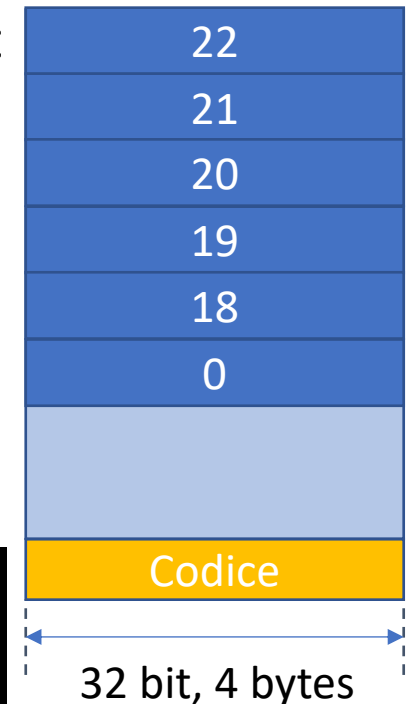
    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC

int i

```
$ ./fig06_01
0      18
```



Accesso tramite indice

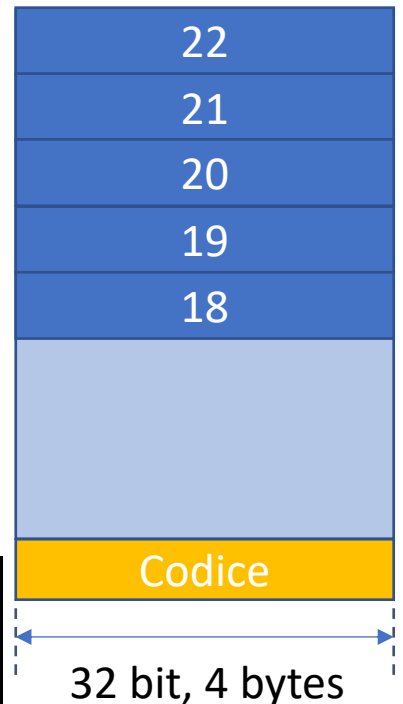
```
#define SIZE_A 5
int main(void) {
    int voti[SIZE_A];

    for (int i = 0; i < SIZE_A; i++) {
        voti[i] = i + 18;
    }

    for (int i = 0; i < SIZE_A; i++) {
        printf("%d\t%d", i, voto[i]);
    }
}
```



int voti[4]@0x3FC
int voti[3]@0x3F8
int voti[2]@0x3F4
int voti[1]@0x3F0
int voti[0]@0x3EC



```
$ ./fig06_01
0      18
1      19
2      20
3      21
4      22
```

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Array predefiniti

- Il linguaggio C permette di definire array tramite una lista di valori noti

```
int voti[5] = {18, 19, 20, 21, 22}
```

- Equivalente a

```
int voti[] = {18, 19, 20, 21, 22}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```



Array predefiniti

- Il linguaggio C permette di definire array tramite una lista di valori noti

```
int voti[5] = {18, 19, 20, 21, 22}
```

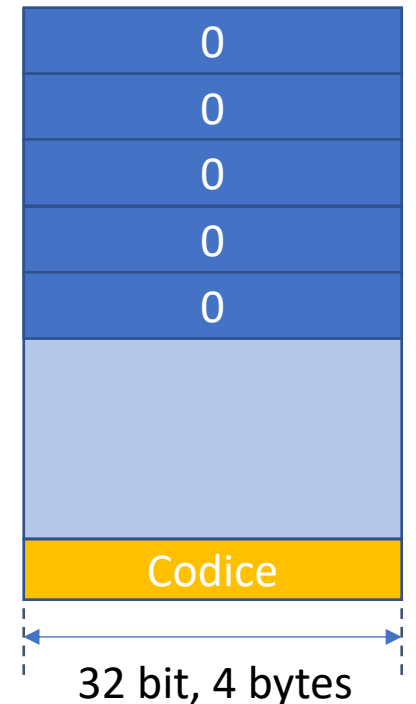
- Equivalente a

```
int voti[] = {18, 19, 20, 21, 22}
```

- Utile per inizializzare un array a zero

```
int voti[5] = {0}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```



Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Buffer overflow

- L'accesso alla cella `SIZE_A`-esima comporta una lettura o scrittura al di fuori dell'area di memoria allocata per l'array
- Nel migliore dei casi, il programma si blocca riportando errore di segmentazione o simili
- Frequentemente, vengono letti dalla memoria dati non corretti che compromettono la correttezza dell'output immediato
- Nel peggiore dei casi, si va a modificare inavvertitamente altre strutture dati senza che il problema si manifesti immediatamente

Buffer overflow

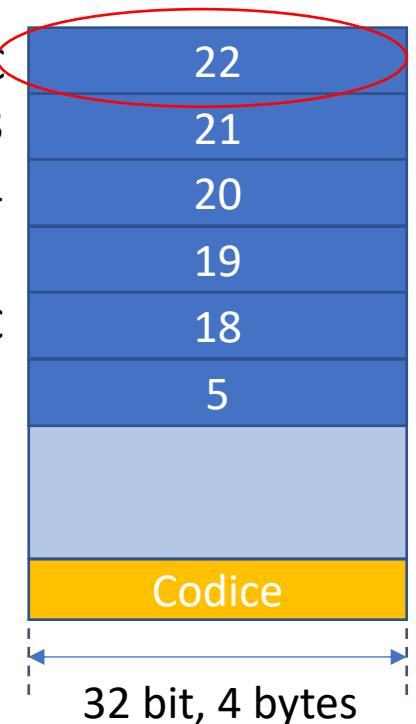


```
#define SIZE_A 5
```

```
int main(void) {  
    int voti[] = {18, 19, 20, 21, 22}  
  
    for (int i = 0; i <= SIZE_A; ++i) {  
        printf("%d\t%d", i, voto[i]);  
    }  
}
```

```
int voti[4]@0x3FC  
int voti[3]@0x3F8  
int voti[2]@0x3F4  
int voti[1]@0x3F0  
int voti[0]@0x3EC
```

int i



```
$ ./fig06_01  
0      18  
1      19  
2      20  
3      21  
4      22
```

Esercizio BARs: for annidati e array

```
1.  // fig06_06.c
2.  // Displaying a bar chart.
3.  #include <stdio.h>
4.  #define SIZE 5
5.
6.  // function main begins program
   execution
7.  int main(void) {
8.      // use initializer list to
   initialize array n
9.      int n[SIZE] = {19, 3, 15, 7,
10.         11};
11.
12.      printf("%s%13s%17s\n",
13.         "Element", "Value", "Bar
   Chart");
14.
15.      for (size_t i = 0; i < SIZE; ++i) {
16.          printf("%7zu%13d%8s", i, n[i], "");
17.
18.          for (int j = 1; j <= n[i]; ++j) {
19.              printf("%c", '*');
20.          }
21.          puts(""); //end a bar with a \n
22.      }
23. }
```

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Passaggio di array come parametri

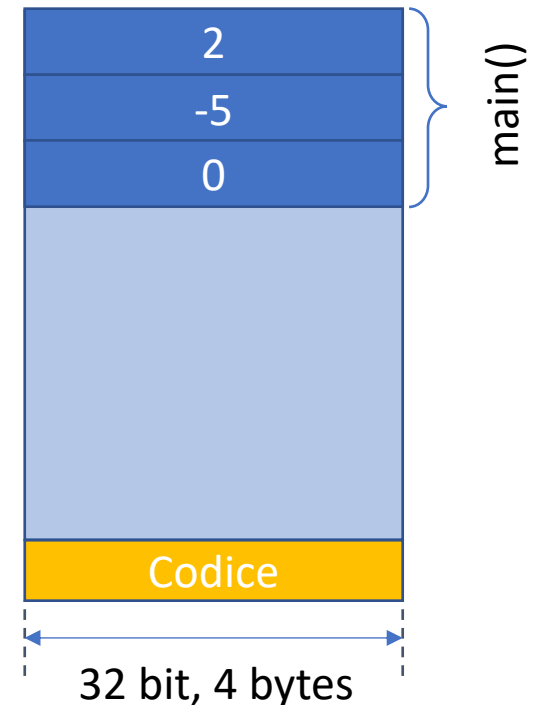
- È caso comune (*aka*, caso d'esame) che un array allocato nel `main()` sia manipolato in una funzione chiamata dal `main()`
- Il passaggio di un array come parametro avviene per riferimento (indiretto), ovvero nello stack è copiato il puntatore al primo elemento dell'array, **non l'array stesso!**
- La funzione chiamata riceve **come primo parametro un puntatore di tipo `int[]`**, che ci permette di accedere al valore degli elementi dell'array senza dereferenziazione
- Come **secondo parametro** passeremo la **dimensione dell'array**

Passaggio di array come parametri

```
#define SIZE_A 3
```

```
→ void main(void) {  
    int a[] = {2, -5, 0};  
    square (a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, a[i]);  
    }  
}  
  
void square (int a[], int lenA) {  
    for (int i = 0; i < lenA; i++) {  
        a[i] = a[i] * a[i];  
    }  
}
```

int a[2]@0x3FC
int a[1]@0x3F8
int a[0]@0x3F4



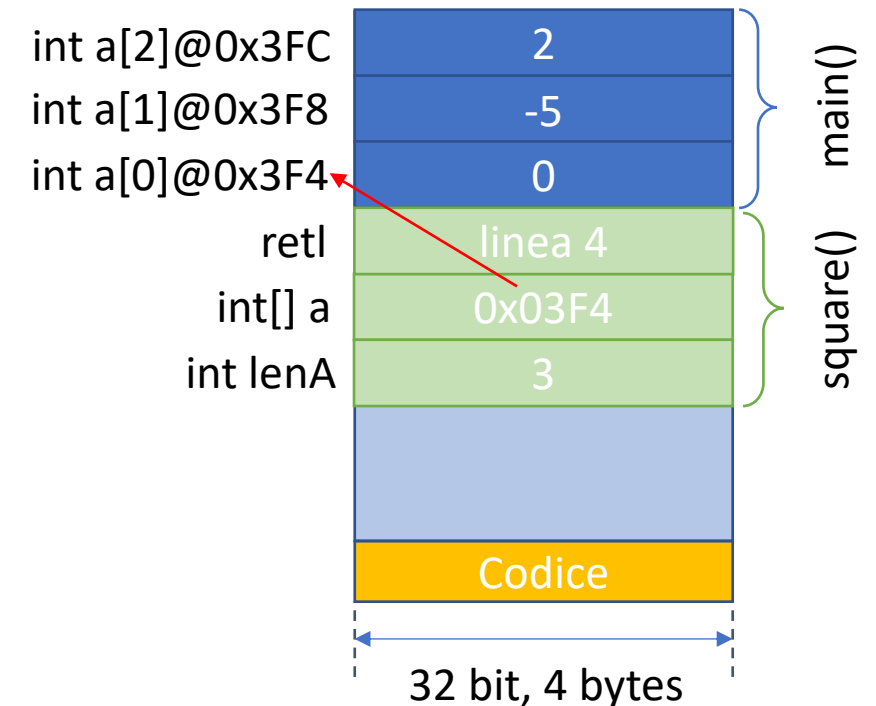
Passaggio di array come parametri

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {2, -5, 0};  
    square (a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, a[i]);  
    }  
}
```

➔

```
void square (int a[], int lenA) {  
    for (int i = 0; i < lenA; i++) {  
        a[i] = a[i] * a[i];  
    }  
}
```

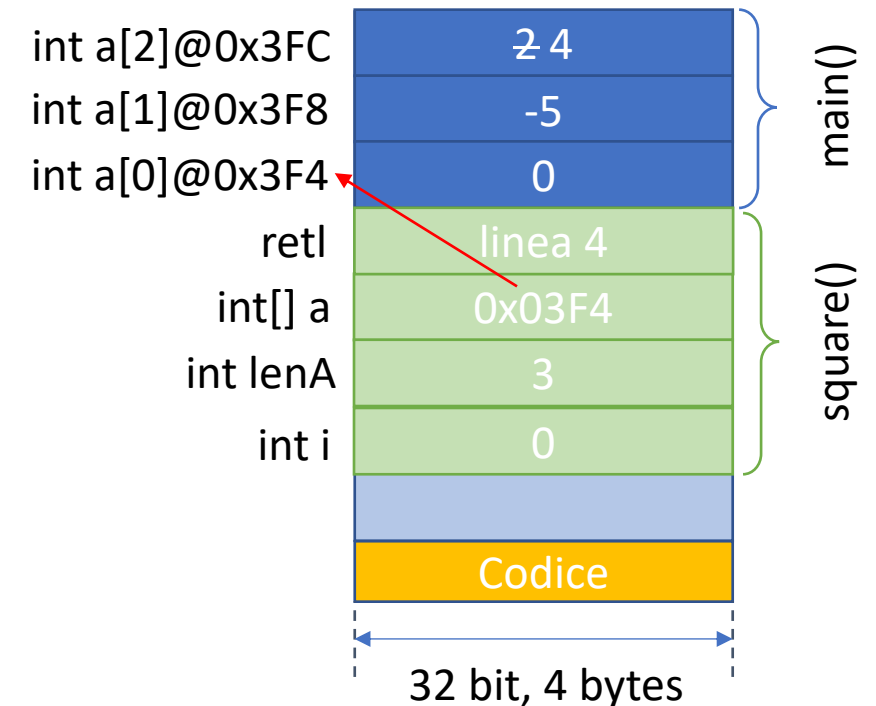


Passaggio di array come parametri

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {2,-5,0};  
    square (a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, a[i]);  
    }  
}
```

```
void square (int a[], int lenA) {  
    for (int i = 0; i < lenA; i++) {  
        a[i] = a[i] * a[i];  
    }  
}
```

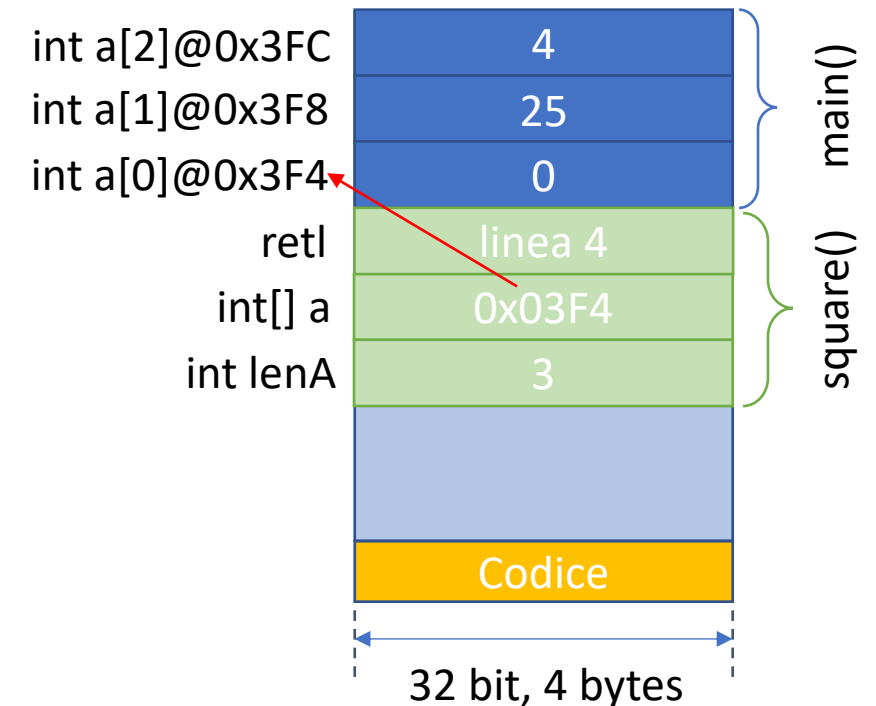


Passaggio di array come parametri

```
#define SIZE_A 3
```

```
void main(void) {  
    int a[] = {2,-5,0};  
    square (a, SIZE_A);  
    for (int i = 0; i < SIZE_A; i++) {  
        printf("%d\t%d", i, a[i]);  
    }  
}
```

```
void square (int a[], int lenA) {  
    for (int i = 0; i < lenA; i++) {  
        a[i] = a[i] * a[i];  
    }  
}
```



array, &array[0], &array

```
1.  // fig06_10.c
2.  // Array name is the same as the address of the array's first element.
3.  #include <stdio.h>
4.
5.  // function main begins program execution
6.  int main(void) {
7.      char array[5] = ""; // define an array of size 5
8.
9.      printf("    array = %p\n&array[0] = %p\n    &array = %p\n",
10.         array, &array[0], &array);
11. }
```

Output:

```
    array = 0031F930
&array[0] = 0031F930
&array = 0031F930
```

6.7 Passing Arrays to Functions

Difference Between Passing an Entire Array and Passing an Array Element

- `fig06_11.c` demonstrates the difference between passing an entire array and passing an individual array element
- We pass array `a` and its size to `modifyArray` (line 24), which multiplies each of `a`'s elements by 2 (lines 45–47)
 - The output shows, `modifyArray` did, indeed, modify `a`'s elements
- We pass `a[3]`'s value to function `modifyElement`
 - The function multiplies its argument by 2 (line 53) and prints the new value
 - When line 39 in `main` displays `a[3]` again, it has **not** been modified because individual array elements are passed by value

6.7 Passing Arrays to Functions `fig06_11.c` **1**

```
1.  // fig06_11.c
2.  // Passing arrays and individual array elements to functions.
3.  #include <stdio.h>
4.  #define SIZE 5
5.
6.  // function prototypes
7.  void modifyArray(int b[], size_t size);
8.  void modifyElement(int e);
9.
```

6.7 Passing Arrays to Functions `fig06_11.c` 2

```
10. // function main begins program execution
11. int main(void) {
12.     int a[SIZE] = {0, 1, 2, 3, 4}; // initialize array a
13.
14.     puts("Effects of passing entire array by reference:\n\nThe "
15.         "values of the original array are:");
16.
17.     // output original array
18.     for (size_t i = 0; i < SIZE; ++i) {
19.         printf("%3d", a[i]);
20.     }
21.
22.     puts(""); // outputs a newline
```

6.7 Passing Arrays to Functions `fig06_11.c` 4

```
23.  
24.    modifyArray(a, SIZE); // pass array a to modifyArray by reference  
25.    puts("The values of the modified array are:");  
26.  
27.    // output modified array  
28.    for (size_t i = 0; i < SIZE; ++i) {  
29.        printf("%3d", a[i]);  
30.    }  
31.
```

6.7 Passing Arrays to Functions `fig06_11.c` 5

```
32.    // output value of a[3]
33.    printf("\n\n\nEffects of passing array element "
34.        "by value:\n\nThe value of a[3] is %d\n", a[3]);
35.
36.    modifyElement(a[3]); // pass array element a[3] by value
37.
38.    // output value of a[3]
39.    printf("The value of a[3] is %d\n", a[3]);
40. }
41.
```

6.7 Passing Arrays to Functions `fig06_11.c` 6

```
42. // in function modifyArray, "b" points to the original array "a" in memory
43. void modifyArray(int b[], size_t size) {
44.     // multiply each array element by 2
45.     for (size_t j = 0; j < size; ++j) {
46.         b[j] *= 2; // actually modifies original array
47.     }
48. }
49.
50. // in function modifyElement, "e" is a local copy of array element
51. // a[3] passed from main
52. void modifyElement(int e) {
53.     e *= 2; // multiply parameter by 2
54.     printf("Value in modifyElement is %d\n", e);
55. }
```

6.7 Passing Arrays to Functions `fig06_11.c` 7

Effects of passing entire array by reference:

The values of the original array are:

0 1 2 3 4

The values of the modified array are:

0 2 4 6 8

Effects of passing array element by value:

The value of `a[3]` is 6

Value in `modifyElement` is 12

The value of `a[3]` is 6

6.7 Passing Arrays to Functions

Using `const` to Prevent Functions from Modifying Array Elements

- When an array parameter is preceded by `const`, the function treats the elements as constants
- This is another example of the principle of least privilege (**privilegio minimo**)
 - A function should not be given the capability to modify an array in the caller unless it's absolutely necessary.

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricerca linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Ricerca lineare

- Scrivere una funzione

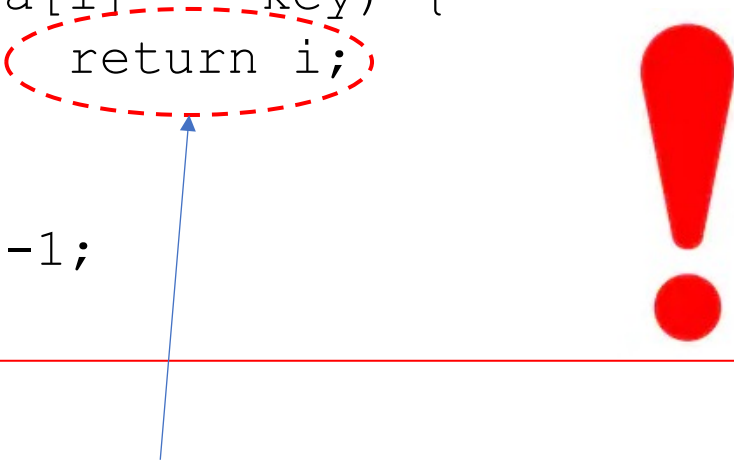
```
int linearSearch(int a[], int lenA, int key)
```

- che, dati in input un array *a*, la relativa lunghezza *lenA* e una chiave di ricerca, ritorni la posizione dell'elemento nell'array, -1 altrimenti
- Il libro di testo (Sezione 6.10) propone una soluzione subottima
 - Interruzione ciclo FOR tramite *return*
 - Due *return* nella funzione
 - Ricerca nell'array non terminata alla prima chiave

Ricerca lineare

- Introduciamo la variabile `ret` per memorizzare il valore ritornato

```
int linearSearch(int a[], int lenA,
int key) {
    for (int i = 0; i < lenA; i++){
        if (a[i] == key) {
            return i;
        }
    }
    return -1;
}
```



Interruzione del ciclo
for() tramite return

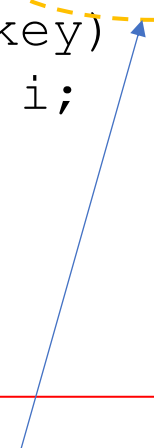
```
int linearSearch(int a[], int lenA,
int key) {
    int ret = -1;
    for (int i = 0; i < lenA; i++){
        if (a[i] == key) {
            ret = i;
        }
    }
    return ret;
}
```



Ricerca lineare

- Introduciamo la variabile `ret` per memorizzare il valore ritornato

```
int linearSearch(int a[], int lenA,
int key) {
    for (int i = 0; i < lenA; i++) {
        if (a[i] == key) {
            return i;
        }
    }
    return -1;
}
```



La ricerca non termina
alla prima chiave trovata

```
int linearSearch(int a[], int lenA,
int key) {
    int ret = -1;
    for (int i = 0; i < lenA && ret == -1; i++) {
        if (a[i] == key) {
            ret = i;
        }
    }
    return ret;
}
```



Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

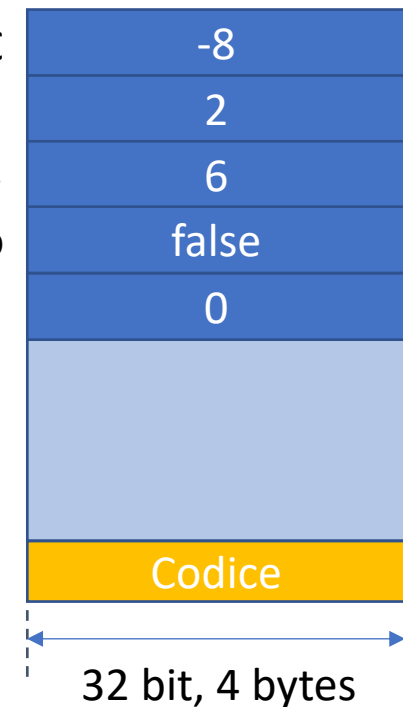
Bubble sort

- Variazione sull'ordinamento di 2 e 3 variabili
- Ordinare gli elementi dell'array in ordine crescente di indice, ovvero l'elemento di indice minore deve contenere il valore minore.
- Scambiare, se necessario, `Array[i]` con `Array[i+1]`
- **Attenzione:** l'indice $(i+1)$ non deve mai uscire dall'array!
- Come codificare la condizione di terminazione del ciclo sugli elementi dell'array?

Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i

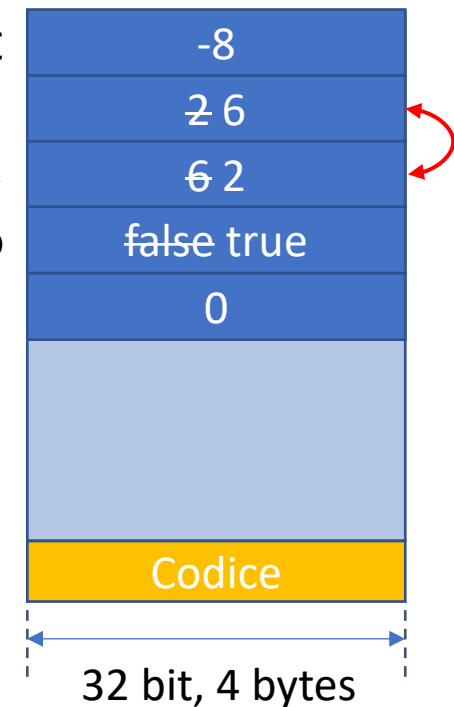


Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

```
void swap (int* pA, int* pB) {
    int temp = *pA;
    *pA = *pB;
    *pB = temp;
}
```

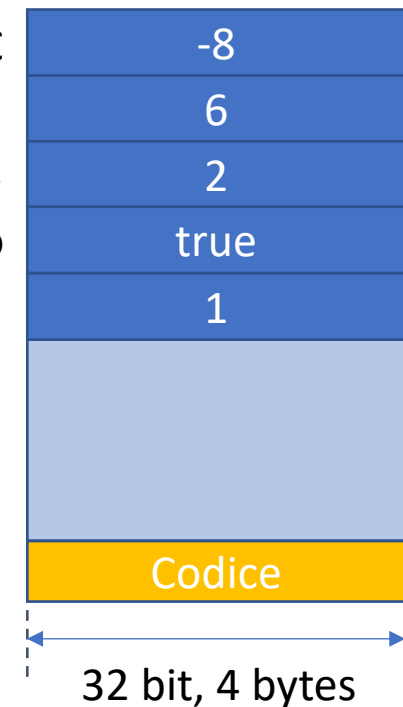
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i

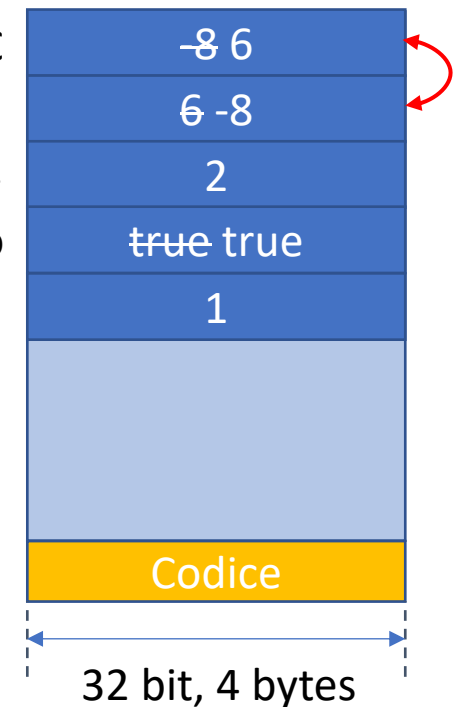


Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

```
void swap (int* pA, int* pB) {
    int temp = *pA;
    *pA = *pB;
    *pB = temp;
}
```

int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
```



```
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

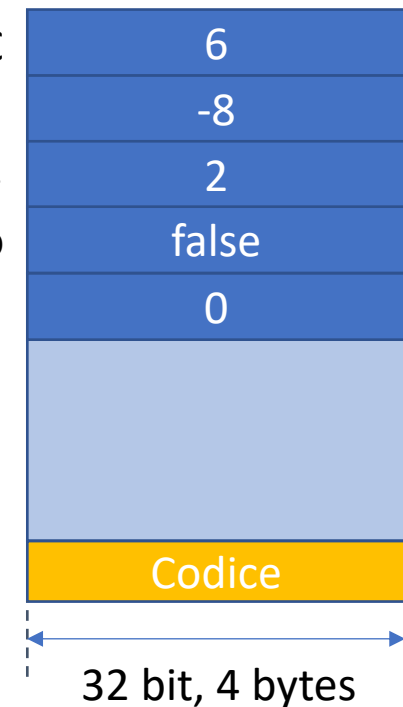
```
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
```



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

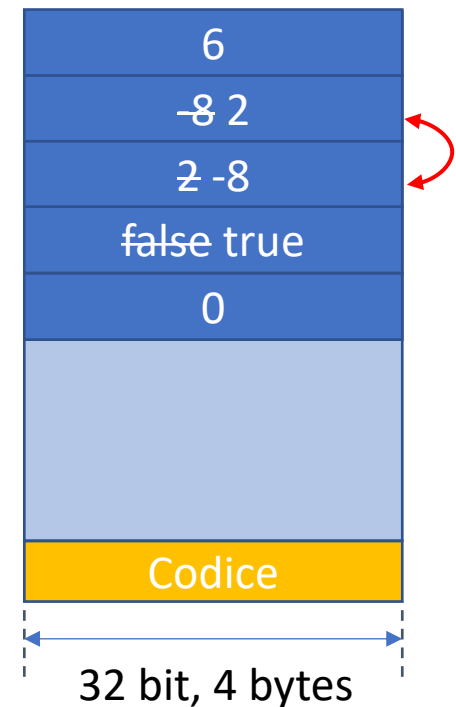
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i



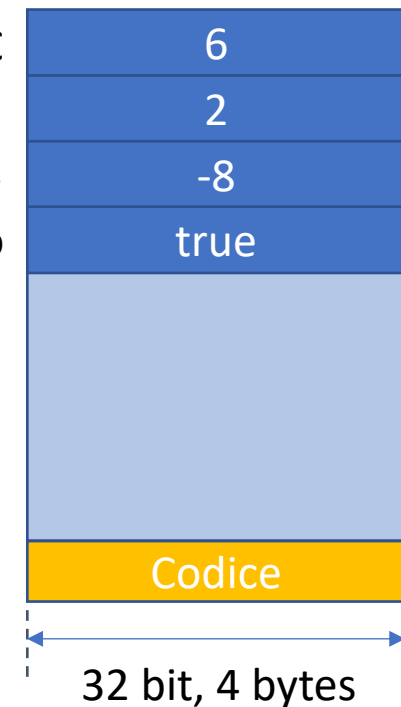
Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
```



```
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

```
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
```



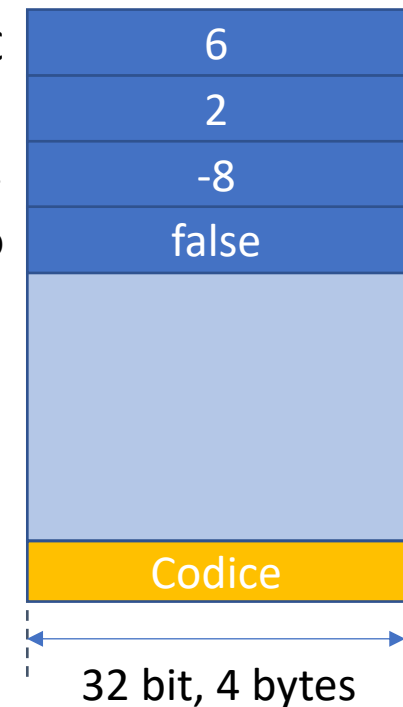
Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
```

→

```
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

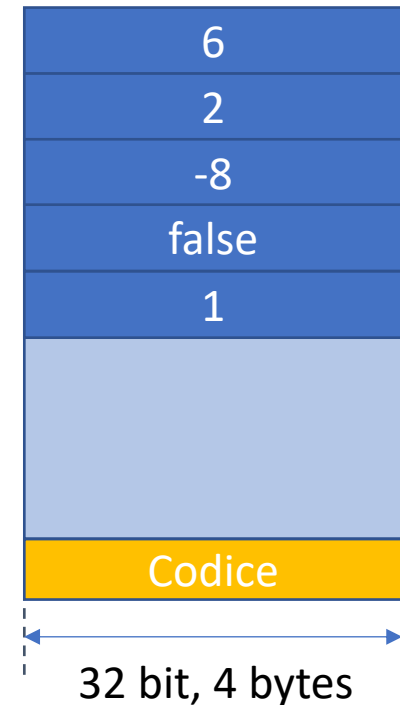
```
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
```



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

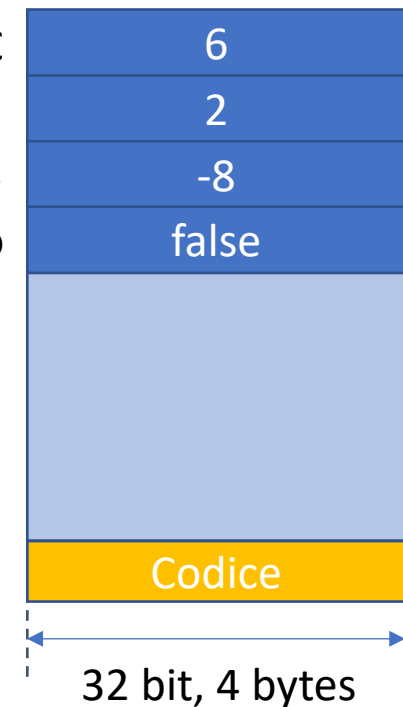
int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo
int i



Bubble sort

```
#define SIZE_A 3
int v[] = {6, 2, -8};
...
int eseguiCiclo = true;
while (eseguiCiclo == true) {
    eseguiCiclo = false;
    for (int i = 0; i < SIZE_A-1; i++) {
        if (v[i] > v[i+1]) {
            swap (&a[i], &a[i+1]);
            eseguiCiclo = true;
        }
    } // end for
} // end while
```

int v[2]@0x3FC
int v[1]@0x3F8
int v[0]@0x3F4
int eseguiCiclo



Section 6.8 Bubble Sort

```
1.  // fig06_12.c
2.  // Sorting an array's values into ascending order.
3.  #include <stdio.h>
4.  #define SIZE 10
5.
6.  // function main begins program execution
7.  int main(void) {
8.      int a[SIZE] = {2, 6, 4, 8, 10, 12, 89, 68, 45, 37};
9.
10.     puts("Data items in original order");
11.
12.     // output original array
13.     for (size_t i = 0; i < SIZE; ++i) {
14.         printf("%4d", a[i]);
15.     }
```

Section 6.8 Bubble Sort

```
17.  // bubble sort
18.  // loop to control number of passes
19.  for (int pass = 1; pass < SIZE; ++pass) {
20.      // loop to control number of comparisons per pass
21.      for (size_t i = 0; i < SIZE - 1; ++i) {
22.          // compare adjacent elements and swap them if first
23.          // element is greater than second element
24.          if (a[i] > a[i + 1]) {
25.              int hold = a[i];
26.              a[i] = a[i + 1];
27.              a[i + 1] = hold;
28.          }
29.      }
30.  }
```

Section 6.8 Bubble Sort

```
31.  
32.     puts("\nData items in ascending order");  
33.  
34.     // output sorted array  
35.     for (size_t i = 0; i < SIZE; ++i) {  
36.         printf("%4d", a[i]);  
37.     }  
38.  
39.     puts("");  
40. }
```

Outline

- Intro Array
- Allocare
- Array e puntatori
- Assegnare
- Leggere
- Accedere con indici
- Array predefiniti
- Buffer Overflow
- Array come parametro formale
- Ricercare linearmente
- Bubble Sort con Array
- Funzione Bubble Sort con Array
- Variable Length Array

Bubble sort come funzione

- Scrivere una funzione `bubbleSort()` che riceva un array *a* come parametro e relativa lunghezza *lenA* come parametri
- La funzione ritorni anche il numero di scambi effettuati

Bubble sort come funziona

```
int bubbleSort(int a[], int lenA) {  
    int ret=0;  
    int eseguiCiclo = true;  
    while (eseguiCiclo == true) {  
        eseguiCiclo = false;  
        for (int i = 0; i < lenA; i++) {  
            if (a[i] > a[i+1]) {  
                swap (&a[i], &a[i+1]);  
                eseguiCiclo = true;  
                ret++;  
            }  
        } // end for  
    } // end while  
    return ret;  
}
```

```
void swap (int* pA, int* pB) {  
    int temp = *pA;  
    *pA = *pB;  
    *pB = temp;  
}
```

